

Desafio: Gerenciador de Tarefas

Construção de uma API REST completa com Java 25 e Spring Boot 4

Modalidade	Entrega	Peso
Individual	Link do repositório no GitHub	Avaliação final da disciplina

1. Contexto

Uma equipe pequena precisa organizar o trabalho do dia a dia e hoje faz isso em planilhas espalhadas. Cada pessoa mantém a sua, ninguém sabe o que já foi concluído e as tarefas não têm dono claro. A equipe decidiu contratar um **serviço back-end** que centralize essa informação e sirva de base para um aplicativo web e um aplicativo móvel que serão desenvolvidos depois.

Você foi contratado para entregar esse serviço. Ele não tem tela: expõe uma **API REST** que qualquer cliente HTTP consegue consumir. O seu compromisso é que a API funcione corretamente, esteja bem organizada em camadas e persista os dados em banco.

2. O que você vai construir

Uma aplicação Spring Boot que permita cadastrar projetos, cadastrar responsáveis e gerenciar o ciclo de vida das tarefas de cada projeto — criar, listar, filtrar, consultar, atualizar e remover — com os dados gravados em banco relacional.

Escopo obrigatório em uma frase

Três entidades persistidas (*Projeto*, *Responsavel*, *Tarefa*), CRUD completo de tarefas, pelo menos um filtro por query string e tratamento adequado de recurso inexistente.

3. Requisitos funcionais

3.1 Obrigatórios

Código	Requisito
RF01	Cadastrar um projeto informando nome e descrição.
RF02	Listar todos os projetos cadastrados.
RF03	Cadastrar um responsável informando nome e e-mail.

Código	Requisito
RF04	Listar todos os responsáveis cadastrados.
RF05	Criar uma tarefa vinculada a um projeto existente, com título, descrição, prioridade e prazo. A tarefa nasce com status <code>NOVA</code> e com a data de criação preenchida pelo sistema.
RF06	Listar todas as tarefas.
RF07	Filtrar tarefas por status e por projeto usando query string, por exemplo <code>GET /tarefas?status=NOVA</code> .
RF08	Consultar uma tarefa específica pelo seu identificador.
RF09	Atualizar os dados de uma tarefa existente, incluindo a troca de status e a atribuição de um responsável.
RF10	Remover uma tarefa.
RF11	Ao mudar o status para <code>CONCLUIDA</code> , o sistema deve preencher automaticamente a data de conclusão.
RF12	Consultar um projeto ou uma tarefa que não existe deve resultar em uma resposta de recurso não encontrado, e não em erro interno do servidor.

3.2 Desafios opcionais (bônus)

Só valem se todo o escopo obrigatório estiver funcionando. Escolha um ou dois — não é necessário fazer todos.

- **Etiquetas (N:N).** Uma tarefa pode ter várias etiquetas e uma etiqueta pode marcar várias tarefas.
- **Comentários (1:N).** Registrar comentários em uma tarefa, com autor e data.
- **Endpoint de ação.** `PUT /tarefas/{id}/concluir` para concluir a tarefa sem enviar o objeto inteiro.
- **Recurso aninhado.** `GET /projetos/{id}/tarefas` para listar as tarefas de um projeto.
- **Paginação e ordenação.** Receber `Pageable` no listar e devolver a página em vez da lista inteira.
- **PostgreSQL em container.** Rodar a aplicação contra o Postgres via Docker, usando um profile separado, sem alterar uma linha de código Java.

4. Requisitos técnicos

Estes pontos serão verificados na avaliação. Eles são o motivo pelo qual a disciplina existe.

Item	O que é exigido
Plataforma	Java 25 e Spring Boot 4.x, projeto gerado pelo Spring Initializr e construído com Maven.

Item	O que é exigido
Dependências	Spring Web, Spring Data JPA e PostgreSQL. Outras podem ser usadas se fizerem sentido.
Organização	Pacotes separados por camada: <code>controller</code> , <code>service</code> , <code>repository</code> e <code>domain</code> .
Injeção	Todas as dependências injetadas por construtor . Não use <code>@Autowired</code> em campo.
Responsabilidades	O controller não conversa com o repositório e não decide regra de negócio. A regra fica no service.
Persistência	Entidades mapeadas com JPA e repositórios estendendo <code>JpaRepository</code> . Pelo menos uma consulta derivada do nome do método.
Enums	Status e prioridade devem ser enums, gravados com <code>@Enumerated(EnumType.STRING)</code> .
Erros	Uma exceção de domínio própria para recurso inexistente, resultando em 404 com mensagem clara.
Verificação	Coleção do Postman (ou arquivo <code>.http</code>) cobrindo todas as operações, versionada no repositório.

Fora do escopo — não faça

Autenticação e autorização. A API fica aberta. Spring Security é assunto de Programação Web Back-end II.

Validação com Bean Validation. Não é necessário anotar os campos com `@NotBlank`, `@Size` e afins — também é conteúdo da disciplina seguinte.

DTOs e conversores. Pode trafegar a entidade diretamente na requisição e na resposta. Sabemos que não é a prática ideal; ela será corrigida na refatoração de Back-end II.

Testes automatizados. A verificação aqui é manual, via Postman.

Front-end. Nenhuma tela é esperada. O entregável é a API.

5. Modelo de dados

O modelo abaixo já está pronto para você — a modelagem relacional é conteúdo da disciplina de Banco de Dados, e aqui ela não é o objeto de avaliação. As entidades em verde-escuro são obrigatórias; as em tom claro, com ligação tracejada, pertencem aos desafios opcionais.

Gerenciador de Tarefas — Modelo Entidade-Relacionamento

Atividade final · Programação Web Back-end I · linha cheia = escopo obrigatório · linha tracejada = extensão opcional

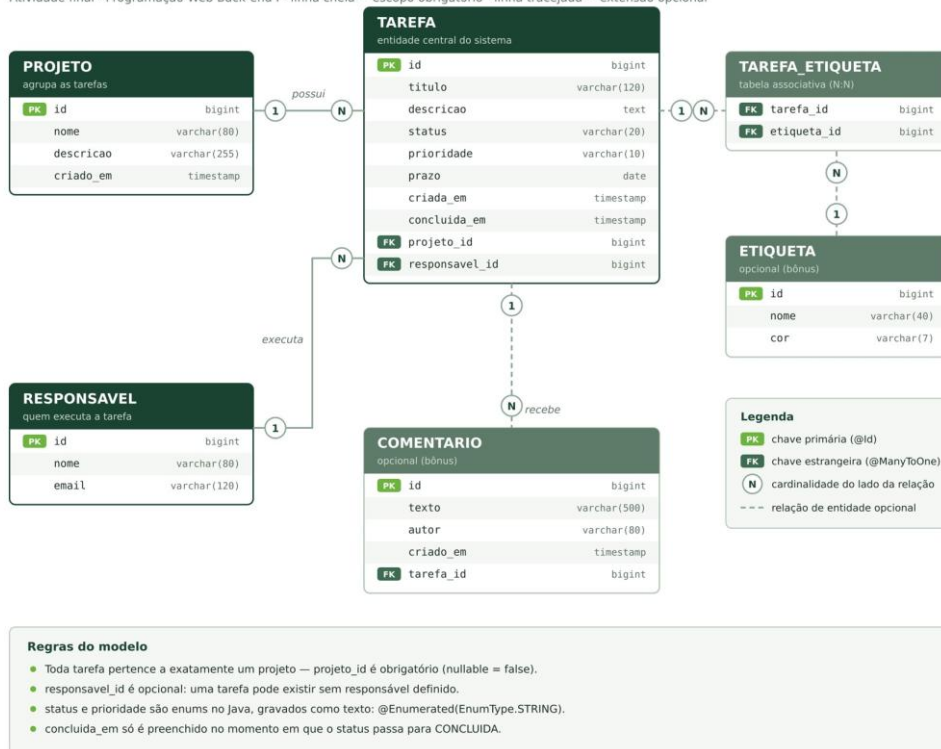


Figura 1 — MER do Gerenciador de Tarefas. As tabelas em tom claro são opcionais.

5.1 Dicionário de dados da entidade TAREFA

Campo	Tipo	Obrigatório	Regra de preenchimento
id	Long	gerado	Chave primária, gerada pelo banco.
titulo	String(120)	sim	Resumo do que precisa ser feito.
descricao	String	não	Detalhamento livre da tarefa.
status	Enum	sim	NOVA, EM_ANDAMENTO, CONCLUIDA ou CANCELADA. Nasce como NOVA.
prioridade	Enum	sim	BAIXA, MEDIA ou ALTA. Padrão sugerido: MEDIA.
prazo	LocalDate	não	Data limite para a conclusão.
criadaEm	LocalDateTime	sim	Preenchida pelo sistema no momento da criação.
concluidaEm	LocalDateTime	não	Preenchida quando o status passa para CONCLUIDA.
projeto	Projeto	sim	Relação @ManyToOne. Toda tarefa pertence a um projeto.

Campo	Tipo	Obrigatório	Regra de preenchimento
responsavel	Responsavel	não	Relação @ManyToOne. A tarefa pode ficar sem responsável.

6. Contrato da API

Respeite estas rotas e estes códigos de status. Eles fazem parte da avaliação.

Método e rota	O que faz	Sucesso	Falha
POST /projetos	Cria um projeto	201	—
GET /projetos	Lista os projetos	200	—
POST /responsaveis	Cria um responsável	201	—
GET /responsaveis	Lista os responsáveis	200	—
POST /tarefas	Cria uma tarefa	201 + Location	—
GET /tarefas	Lista tarefas, com filtros opcionais	200	—
GET /tarefas/{id}	Consulta uma tarefa	200	404
PUT /tarefas/{id}	Atualiza uma tarefa	200	404
DELETE /tarefas/{id}	Remove uma tarefa	204	404

6.1 Exemplo de criação de tarefa

Requisição — POST /tarefas

```
{
  "titulo": "Revisar o contrato do fornecedor",
  "descricao": "Conferir clausulas de reajuste antes da renovacao",
  "prioridade": "ALTA",
  "prazo": "2026-09-30",
  "projeto": { "id": 1 },
  "responsavel": { "id": 2 }
}
```

Resposta — 201 Created, com o cabeçalho Location: /tarefas/7

```
{
  "id": 7,
  "titulo": "Revisar o contrato do fornecedor",
  "descricao": "Conferir clausulas de reajuste antes da renovacao",
  "status": "NOVA",
  "prioridade": "ALTA",
  "prazo": "2026-09-30",
  "criadaEm": "2026-09-01T14:20:11",
  "concluidaEm": null
}
```

6.2 Exemplo de recurso inexistente

Requisição — GET /tarefas/999 • **Resposta** — 404 Not Found

```
{
  "erro": "Tarefa nao encontrada: 999",
  "momento": "2026-09-01T14:31:02Z"
}
```

O formato do corpo do erro é livre, mas deve ser **consistente** em toda a API e nunca expor a stack trace da exceção.

7. Roteiro de verificação

Antes de entregar, execute esta sequência na sua coleção do Postman. Se ela passar inteira, o núcleo do trabalho está fechado.

1. Criar um projeto e conferir o status 201.
2. Criar um responsável e conferir o status 201.
3. Criar três tarefas no projeto, com prioridades diferentes.
4. Listar as tarefas e conferir que as três aparecem, com status NOVA.
5. Filtrar por status e conferir que o filtro realmente reduz o resultado.
6. Consultar uma tarefa pelo id e conferir o corpo devolvido.
7. Atualizar uma tarefa mudando o status para CONCLUIDA e conferir que a data de conclusão foi preenchida.
8. Atribuir um responsável a uma tarefa e conferir o resultado na consulta.
9. Remover uma tarefa e conferir o status 204.
10. Consultar a tarefa removida e conferir que a resposta é 404, com mensagem legível.

8. Entrega

- **Repositório público no GitHub**, com o link enviado pelo canal combinado da turma.
- **README.md** com: descrição do projeto, requisitos de ambiente, como rodar a aplicação, qual banco foi usado e a lista de endpoints.

9. Critérios de avaliação

Critério	Peso	O que se espera
Funcionamento do CRUD	30%	As operações respondem corretamente, com os códigos de status definidos no contrato.
Organização em camadas	25%	Responsabilidades separadas, injeção por construtor, sem regra de negócio no controller.

Critério	Peso	O que se espera
Modelagem e persistência	20%	Entidades bem mapeadas, relacionamentos corretos e ao menos uma consulta derivada.
Tratamento de erros	15%	Recurso inexistente devolve 404 com mensagem clara, e não 500 com stack trace.
Entrega e documentação	10%	Commits descritivos, README completo e coleção de testes versionada.

10. Erros comuns (leia antes de começar)

- **Classes fora do pacote raiz.** Tudo precisa ficar abaixo do pacote da classe `Application`; o component scan não encontra o que estiver fora e a injeção falha.
- **Entidade sem construtor sem argumentos.** O JPA exige um construtor protegido ou público sem parâmetros. Sem ele, a aplicação nem sobe.
- **Esquecer o `@RequestBody`.** No POST e no PUT, sem essa anotação o objeto chega com todos os campos nulos e o erro parece vir do banco.
- **Enum sem `@Enumerated(EnumType.STRING)`.** O JPA passa a gravar a posição do enum; reordenar as constantes depois corrompe os dados existentes.
- **Devolver 500 no lugar de 404.** Se `findById` volta vazio e você chama `.get()` direto, o cliente recebe erro interno. Use `orElseThrow` com a sua exceção de domínio.
- **Regra de negócio no controller.** Preencher a data de conclusão dentro do controller é o erro mais frequente. Essa decisão pertence ao service.
- **Loop infinito no JSON.** Relacionamento bidirecional sem cuidado faz o Jackson entrar em recursão. A saída mais simples nesta disciplina é manter a relação apenas no lado `@ManyToOne`.